MP2

Descripción del proyecto

Lenguaje: Java 21 · **Modalidad:** grupos de 3 personas · **Entrega:** repositorio GitHub público; la entrega final debe estar en la rama main y los integrantes deben estar en el README (el que no esté tendrá una penalización fuerte).

El MP2 continúa la simulación de Pokémon TCG Pocket (expansión Genetic Apex) construida en el MP1, con dos cambios: el juego deja la consola y pasa a una **interfaz gráfica de escritorio en Java Swing**, y se agrega la mecánica de **evolución**.

Todas las reglas del MP1 siguen vigentes (RF1 a RF4) salvo lo que este enunciado modifique explícitamente. La idea es reutilizar el modelo del MP1, no reescribirlo: si su diseño OOP estaba bien hecho, la lógica del juego no debería cambiar al cambiar la interfaz.

Objetivos de aprendizaje

- Construir una GUI con Swing: contenedores, layouts, componentes y manejo de eventos con listeners.
- Separar la lógica del juego (modelo) de la presentación (vista), al estilo MVC.
- Extender una jerarquía de clases existente (nuevas etapas de Pokémon) sin romper lo que ya funciona.

Nota importante: igual que en el MP1, NO HAGA NADA ADICIONAL A LO QUE SE PIDE AQUÍ. Las reglas se seguirán agregando en entregas futuras. En especial no hagan el modelo vista controlador, o el proyecto ***NO*** será valido

Requisitos funcionales

RF6 Cartas de evolución

Se agregan exactamente las siguientes 7 cartas. Además de los atributos de un Pokémon del RF1, cada una tiene su **etapa** (Fase 1 o Fase 2) y el **nombre del Pokémon del que evoluciona**. Ninguna es Pokémon-ex: al ser derrotadas otorgan 1 punto de premio.

#	Pokémon	Etapa	Evoluciona de	Tipo	PS	Retirada	Ataque
11	Raichu	Fase 1	Pikachu	Eléctrico	100	1	Trueno (2 Eléctrico): 60 daño
12	Charmeleon	Fase 1	Charmander	Fuego	90	2	Garra Ígnea (2 Fuego): 50 daño
13	Charizard	Fase 2	Charmeleon	Fuego	150	2	Giro Fuego (4 Fuego): 120 daño

#	Pokémon	Etapa	Evolucion a de	Tipo	PS	Retirada	Ataque
14	Wartortle	Fase 1	Squirtle	Agua	80	1	Salpicadura (2 Agua): 40 daño
15	Blastoise	Fase 2	Wartortle	Agua	150	3	Hidrobomba (3 Agua): 100 daño
16	Wigglytuff	Fase 1	Jigglypuff	Incoloro	110	2	Bofetón (2 Incoloro): 50 daño
17	Gyarados	Fase 1	Magikarp	Agua	150	4	Hiperrayo (4 Agua): 100 daño

RF7 Mecánica de evolución

Cómo se evoluciona:

- Durante su turno, el jugador puede colocar una carta de evolución de su mano sobre uno de sus Pokémon en juego (activo o banco) cuyo nombre coincida con "Evoluciona de".
- Una Fase 1 se coloca sobre su Pokémon básico; una Fase 2 solo sobre su Fase 1. No se puede saltar de básico a Fase 2.
- Puede evolucionar varios Pokémon distintos en un mismo turno, y puede atacar después de evolucionar.

Restricciones:

- Ningún jugador puede evolucionar durante su primer turno.
- No se puede evolucionar un Pokémon que entró en juego en ese mismo turno.
- Cada Pokémon puede evolucionar solo una vez por turno (de básico a Fase 2 toma al menos dos turnos).

Qué pasa al evolucionar:

- El Pokémon conserva las energías adjuntas y el daño recibido.
- Sus PS máximos pasan a ser los de la evolución: PS actuales = PS máximos de la evolución − daño acumulado. Ejemplo: Charmander con 20/60 PS (40 de daño) evoluciona a Charmeleon y queda con 50/90 PS.
- Tipo, ataques y costo de retirada pasan a ser los de la evolución; los ataques de la etapa anterior ya no se pueden usar.
- Si un Pokémon evolucionado es derrotado, todas sus cartas (la básica y sus evoluciones) y sus energías salen del campo.

RF8 Cambios a las reglas del MP1

- **Mazo (RF4):** sigue siendo de exactamente 20 cartas con máximo 2 copias por carta, ahora usando las cartas del RF1 y del RF6. El mínimo de 6 Pokémon básicos se mantiene; las cartas de evolución no cuentan como básicos.

- **Preparación y banco (RF2):** solo los Pokémon básicos pueden ponerse como activo o en el banco desde la mano. Una carta de evolución solo entra en juego evolucionando.
- **Mulligan (RF4):** se repite si la mano inicial no tiene ningún Pokémon básico; una carta de evolución no evita el mulligan.
- **Poké Ball:** sigue buscando solo un Pokémon básico, no una evolución.
- **Acciones del turno (RF2):** se agrega la acción "Evolucionar Pokémon" a las ya existentes.

RF9 Interfaz gráfica (reemplaza el RF5)

El juego se maneja completamente con la GUI: no se permite leer datos por consola (Scanner, System.in). Los dos jugadores juegan en el mismo computador, por turnos.

Ventana de inicio

- Campos de texto para el nombre de cada entrenador.
- Selección del mazo de cada jugador (por ejemplo, con un JComboBox de mazos predefinidos o una pantalla para armarlo, según lo que implementen).
- Botón para iniciar la partida. Si los datos no son válidos (nombre vacío, mazo inválido), se muestra un mensaje de error y no se avanza.

Tablero de juego debe mostrar en todo momento la misma información que la pantalla de consola del MP1:

- Turno actual y nombre del jugador que juega.
- Campo del rival y campo propio: Pokémon activo y banco, cada uno con nombre, etapa, PS actuales/máximos (con una barra, por ejemplo JProgressBar) y energías adjuntas.
- Puntos de premio de ambos jugadores (x/3) y cantidad de cartas restantes en cada mazo.
- Zona de energía del turno (tipo y si ya se usó).
- Mano del jugador actual, donde cada carta es un componente que se puede seleccionar (por ejemplo, un JButton por carta).
- Un registro de eventos (por ejemplo, JTextArea con scroll) que narre lo que pasa: ataques, daño con debilidad, evoluciones, Pokémon derrotados, cartas jugadas.

Acciones

- Las mismas acciones del MP1 más "Evolucionar Pokémon", disparadas con botones o clics sobre las cartas, no con menús numerados.
- Cuando una acción requiere elegir (a qué Pokémon adjuntar energía, qué Pokémon evolucionar, qué ataque usar, qué Pokémon del banco pasa a activo), la elección se hace en la GUI (diálogos, clic sobre el Pokémon, etc.).
- Las acciones que no se pueden hacer en ese momento deben estar deshabilitadas o mostrar un mensaje claro de por qué no (por ejemplo, "Pikachu no tiene la energía necesaria para Trueno" o "No puedes evolucionar en tu primer turno").

Cambio de turno

- Entre turnos se muestra una pantalla o diálogo de transición ("Turno de Gary presiona Continuar") para que un jugador no vea la mano del otro.

Fin de la partida

- Se anuncia al ganador con un mensaje temático que indique la condición de victoria, con la opción de jugar otra partida o salir.

Restricciones técnicas y entregables

Restricciones técnicas

- No se permite el uso de librerías externas. Solo Java SE estándar; la GUI debe hacerse con Swing (javax.swing y java.awt). JavaFX no está incluido en el JDK 21, así que no se permite.
- No se permiten constructores visuales que generen código automáticamente (por ejemplo, el editor de formularios de NetBeans). La interfaz se escribe a mano.
- **Separación modelo–vista:** las clases del modelo (cartas, jugador, mazo, campo, reglas del turno) no pueden importar nada de javax.swing ni java.awt. La vista llama al modelo y muestra su estado; las reglas del juego no se validan dentro de los listeners.
- **Evolución en el diseño OOP:** las etapas (básico, Fase 1, Fase 2) deben reflejarse en la jerarquía de clases, reutilizando lo que ya tenían del MP1. La decisión de diseño es suya, pero deben poder justificarla en la sustentación.
- Los mazos pueden estar hardcodeados como configuración inicial, pero la lógica del juego debe funcionar con cualquier mazo válido.
- El uso de imágenes para las cartas es opcional y no da puntos extra; si las usan, deben estar dentro del repositorio y cargarse con rutas relativas.

Entregables

1. Repositorio GitHub público con historial de commits y PRs (ojo que esto vale mucho). Se puede continuar el repositorio del MP1 o crear uno nuevo.
2. Archivo README.md con: integrantes, descripción del juego, instrucciones de ejecución y al menos una captura de pantalla del tablero.

Rúbrica de evaluación

Criterio	Descripción	Puntos
Diseño OOP	Uso correcto y justificado de herencia, polimorfismo, encapsulamiento e interfaces. La evolución se integra a la jerarquía sin duplicar lógica. El modelo está separado de la vista.	25
Funcionalidad	El juego completa duelos de principio a fin. Combate, puntos de premio, entrenadores y evolución (con sus restricciones) funcionan correctamente.	35
Interfaz gráfica	La GUI muestra todo el estado del juego, las acciones se hacen con eventos, las acciones inválidas se bloquean o explican, y hay transición entre turnos.	20
Calidad de GitHub	Uso correcto en general (ramas, commits, PRs, etc.).	20

Criterio	Descripción	Puntos
TOTAL		**100**

Penalizaciones

- Entrega tardía: −15 puntos por día hábil de retraso (máximo 2 días; después no se recibe).
- Código con errores de compilación: la entrega no será calificada.
- Entrega no está en la rama main: penalización de 5 puntos.
- El juego pide datos por consola en lugar de la GUI: penalización de 10 puntos.
- Plagio o uso de IA generativa detectado: nota de 0.
- Habrá una sustentación individual, en la cual se le preguntará por cualquier parte del código o de la funcionalidad. No es válido decir "no hice esta parte"; debe estar preparado para cualquier pregunta. La idea es validar que sí hizo el proyecto y lo entiende, no es algo para "corchar". Si no contesta nada de lo que se le pregunte y se le nota absolutamente perdido, su nota puede ser tan baja como 0.0, cosa que espero no pase con nadie.